CHAPTER 17

Streaming, Vision, Files and Documents

In this chapter

Introduction: The Answer That Was Only Two-Thirds True . . . 3

17.1 Streaming Is a Delivery Mode, Not a Different API . . . 4

17.2 The Event Sequence . . . 5

17.3 Where stop_reason Lives, and Why It Matters . . . 6

17.4 The Four Delta Types . . . 7

17.5 Chunky Delivery, and the Stalls You Must Design For . . . 8

17.6 Fine-Grained Tool Streaming . . . 8

17.7 Errors Arrive Inside a 200 . . . 9

17.8 The 21,333 Rule . . . 10

17.9 Streaming Is Perception, Not Physics . . . 10

17.10 Images: Patches, Not Pixels . . . 10

17.11 The Three Source Types . . . 11

17.12 The Limits, in One Place . . . 12

17.13 What Vision Is Reliable For — Chapter 10's Triage, Applied . . . 13

17.14 The Files API . . . 14

17.15 Documents: A PDF Is Text and Images . . . 15

17.16 Documents and Citations . . . 16

17.17 Visual Analogies . . . 16

17.18 Architecture: Where the Stream Terminates . . . 17

17.19 Examples: Four Requests . . . 18

17.20 Code Walkthrough: A Parser, and the Thing It Refuses to Do . . . 18

17.21 Mini Project: An SSE Reassembler and a Price List for Pictures . . . 20

17.22 Panel A — One Frame, and the Field That Is Not In It . . . 21

17.23 Panel B — The Headline: Reassembly Equals the Direct Call . . . 22

17.24 Panel C — The Naive Accumulator, Failing Twice . . . 22

17.25 Panel D — Ninety-Three Cut Points . . . 23

17.26 Panel E — The Official Table, Recomputed . . . 24

17.27 Panel F — A Twenty-Page PDF, Priced . . . 24

17.28 Panel G — The Files API Does Not Do What You Think . . . 25

17.29 Evaluation and Deployment 26
17.30 Hands-on Labs 26
17.31 Lab Solutions 27
17.32 Best Practices 27
17.33 Common Mistakes 28
17.34 Performance Considerations 30
17.35 Security Considerations 30
17.36 Real Production Examples 30
17.37 Certification Notes 31
17.38 Twenty Practice Questions 31
17.39 Answers and Explanations 32
17.40 Ten Practical Exercises 33
17.41 Five Challenge Labs 33
17.42 Ten Interview Questions 34
17.43 Summary 34
17.44 Glossary 36
17.45 Further Reading 37

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Set `stream: true` and name **every event type** the Messages API emits, in the order it emits them.
2. Explain why `stop_reason` **is** `null` **in** `message_start` and arrives in `message_delta`, and what that means for a client that renders as it reads.
3. Reassemble a streamed response into the **exact object** the non-streaming call returns, and say why that reassembly is the client's job and nobody else's.
4. Identify the four **delta types** — `text_delta`, `thinking_delta`, `signature_delta`, `input_json_delta` — and accumulate each one correctly, keyed by `index`.
5. Decide when **fine-grained tool streaming** is worth its cost, and handle the invalid JSON it can produce.
6. Price an image from first principles: **patches, not pixels**, and the two resolution tiers that decide how many of them you pay for.
7. Choose correctly between a `base64` source, a `url` source and a `file_id` from the Files API, and say precisely which problem each one solves.
8. Estimate what a **PDF** costs before you send it, and explain why a page is billed as text *and* image rather than one or the other.

NOTE

Prerequisites

Chapter 16 above all: this chapter is its direct sequel and assumes the request body, the typed content blocks, all seven `stop_reason` values, and `wire.py`. **Chapter 6** for tokens as the unit of cost. **Chapter 8** for citations. **Chapter 10** for the capability triage — information, exact execution, external state, checkability. **Chapter 11** for the rate card, the cache multipliers, and the finding that decode dominates latency. **Chapter 13** for `display: "omitted"`. **Chapter 14** for the per-model matrix.

Chapter 16 deferred four things by name and promised them here: the server-sent-event form of the endpoint, images beyond their bare JSON shape, the Files API, and documents. This chapter pays all four debts. Thinking as an API surface — budgets, effort, signatures, interleaving — is Chapter 18, so thinking appears here only where a stream makes it unavoidable.

Introduction: The Answer That Was Only Two-Thirds True

A support widget on a retail site. The model reads an order record and answers questions about it. It streams, because streaming looks fast and the product manager asked for the typing effect.

One Tuesday a customer asks whether a return is still possible. The widget types out:

> Yes, this order is still within the returns window. You can start a return from your account page

and stops. No full stop, no error, no red banner. The customer starts a return. The return is rejected, because the sentence the model was in the middle of writing was:

> … You can start a return from your account page **for the two items marked "standard". The engraved item is personalised and is excluded from the returns policy.**

The connection had dropped mid-stream. Twelve hundred bytes never arrived. The widget's accumulator had concatenated every `text_delta` it received and rendered the result, exactly as it had been written to do. There was no bug in the sense of a wrong line; there was a missing line. Nothing in the code ever asked the question *did this stream finish?*

Here is the uncomfortable arithmetic, and section 17.25 measures it on six recorded streams. Cut those streams at every event boundary — 93 cut points in total — and a strict reassembler refuses **87** of them, correctly, because 87 of them are not a finished message. The naive accumulator refuses **0**. Of the 93, **52** hand a user non-empty text with no signal at all that anything is missing: 29 truncated mid-sentence, 6 with two separate content blocks glued into one, and 17 that contain every character of the answer but cannot prove it, because the field that says why generation stopped arrives second from last and never came.

And this: across those 93 cut points, `stop_reason` is known at exactly **12** of them. For **87 per cent** of a stream's life, a client that is already showing text to a user does not yet know whether that text is an answer, a truncation, or a refusal.

That is the whole lesson of the streaming half of this chapter, and it is a variation on Chapter 16's. There, the mistake was reading `content[0].text` — assuming a shape. Here, the mistake is accumulating deltas — assuming an *ending*. The server sends edits. The message is yours to build, and yours to certify as finished.

The second half of the chapter is about what you send *in*, rather than what comes back: images, files and documents. It ends in money, because that is where this book's mechanical facts always end. A twenty-page PDF that costs 20p as plain text costs **48p** as a PDF, and the reason is one sentence of documentation that most teams read past: each page is supplied to the model as text *and* as an image, and you pay for both.

17.1 Streaming Is a Delivery Mode, Not a Different API

Add one field to the request body from Chapter 16:

```
{"model": "claude-opus-5", "max_tokens": 1024,
 "messages": [{"role": "user", "content": "Hello"}],
 "stream": true}
```

Everything else is unchanged. Same endpoint, same three headers, same required fields, same `system` as a top-level field, same seven stop reasons. What changes is the response: instead of one JSON document, the connection stays open and the server writes a sequence of **server-sent events** (SSE) until the message is finished.

DEFINITION

Server-sent events (SSE)

A one-directional streaming format carried over ordinary HTTP. The server writes text frames; each frame is a run of `field: value` lines terminated by a blank line. The two fields that matter here are `event`, which names the frame, and `data`, which carries a JSON body. A line beginning with a colon is a comment and is ignored. SSE is a W3C standard, not an Anthropic invention, and any SSE client can read the stream.

A single frame looks exactly like this:

```
event: content_block_delta
data: {"type": "content_block_delta", "index": 0,
       "delta": {"type": "text_delta", "text": "Hello"}}
```

Note the redundancy: the event *name* appears on the `event:` line and the same string appears again as `type` inside the JSON. That redundancy is deliberate and useful — a client that reads only the JSON bodies (some HTTP libraries hand you only those) can still dispatch correctly.

The single most important consequence of streaming is stated most simply as a negative. **The API no longer gives you a message. It gives you instructions for building one.** Everything that follows is a consequence of that sentence.

17.2 The Event Sequence

The documented flow, in order:

1. `message_start` — a `Message` object with `content: []`. Empty. This is the envelope.
2. A series of **content blocks**. Each one is a `content_block_start`, then zero or more `content_block_delta` events, then a `content_block_stop`. Every one of these three carries an `index`, which is the block's position in the final `content` array.
3. One or more `message_delta` events, carrying top-level changes to the message — most importantly `stop_reason` — and the cumulative `usage`.
4. A final `message_stop`.

Two more may appear anywhere:

- `ping` — a keep-alive. It carries nothing. Ignore it, but do not treat an unrecognised event as fatal, because ping is the proof that unrecognised events happen.
- `error` — an error delivered *inside* a 200 response. Section 17.7.

One documented exception to the block pattern: during a **server-side fallback** response, a `fallback` content block arrives at each model boundary as a `content_block_start` and `content_block_stop` pair **with no deltas in between**. A reassembler that assumes every block has at least one delta will not crash, but a *renderer* that assumes it will draw an empty bubble.

```
time ->

message_start          {"content": [], "stop_reason": null}
   |
   |  content_block_start   index 0   {"type":"thinking", ...}
   |     content_block_delta  index 0  thinking_delta
   |     content_block_delta  index 0  thinking_delta
   |     content_block_delta  index 0  signature_delta   <-- always last
   |  content_block_stop    index 0
   |
   |  content_block_start   index 1   {"type":"text","text":""}
   |     content_block_delta  index 1  text_delta
   |     ping                                <-- may appear anywhere
   |     content_block_delta  index 1  text_delta
   |  content_block_stop    index 1
   |
message_delta          {"stop_reason":"end_turn"}  usage (CUMULATIVE)
message_stop

Everything above the message_delta line is content.
Everything you need to decide what to DO is on that line.
```

Figure 17.1 — The event sequence for a two-block response

17.3 Where `stop_reason` Lives, and Why It Matters

In `message_start`, `stop_reason` is present and `null`. It is filled in by `message_delta`, the second-to-last event.

This is not an accident of implementation; it could not be otherwise. The server does not know why generation will stop until it stops. But the consequence for your code is severe, and it is the point at which most streaming clients are wrong:

Every character you render before `message_delta` is rendered on the assumption that the message will end well. If it ends at `max_tokens`, the text you have already shown is a truncation. If it ends at `refusal`, the text you have already shown should probably not have been shown at all. If it ends at `tool_use`, the text you have already shown is a preamble to an action that has not happened yet.

Chapter 16 established that `stop_reason` is the control-flow branch. Streaming adds a timing constraint to it: **the branch arrives after the content it governs.** Design accordingly. In practice this means one of three postures:

Posture	What you do	When it fits
Optimistic render	Show deltas as they arrive; on a bad `stop_reason`, retract or annotate.	Chat UIs where the user watches.
Buffer and gate	Accumulate silently; render nothing until `message_stop`.	Anything with a compliance or correctness bar.
Progressive commit	Render, but never let downstream systems act until `message_stop`.	Agents. Always agents.

The widget in the introduction chose the first posture without knowing it had made a choice, and without implementing the "retract or annotate" half of it.

> WARNING
>
> **The one-line rule**
>
> Never let a *side effect* — a database write, an email, a tool invocation, a payment — be triggered by anything except a stream that reached `message_stop` with a `stop_reason` you have explicitly handled. Rendering is reversible. Actions are not. This is the streaming form of the book's running thread: **the model proposes, your system disposes.**

17.4 The Four Delta Types

Every `content_block_delta` carries a `delta` whose `type` tells you how to apply it. There are four, and each one accumulates into a different field of a different kind of block.

Delta type	Field it builds	Block type	How to apply
`text_delta`	`text`	`text`	Append the string.
`thinking_delta`	`thinking`	`thinking`	Append the string.
`signature_delta`	`signature`	`thinking`	Arrives once, just before `content_block_stop`.
`input_json_delta`	`input`	`tool_use`, `server_tool_use`	Append `partial_json` to a **string buffer**; parse at `content_block_stop`.

Three details separate a correct accumulator from a plausible one.

First: `index` is not decoration. Each block event carries the index of the block it belongs to. Your buffers must be keyed by it. The official accumulation contract in the documentation is written as `tool_inputs: dict[int, str] # index -> accumulated JSON string`, and that dictionary is the entire point. Section 17.24 shows what happens to a buffer that is a single string instead: two parallel tool calls produce `{"city":{"city": "Paris"} "London"}`, which is not JSON, and two separate text blocks produce one sentence with the join hidden inside a full stop, which is worse because it parses fine.

Second: `tool_use.input` starts as `{}` and is built from strings. The `content_block_start` for a `tool_use` block carries `"input": {}` — an empty *object*. The deltas carry `partial_json` — *strings*. The final `input` is an object again. The type changes twice on purpose: the empty object marks the slot in the content array; the strings build the value. Code that tries to merge each delta into the object will fail immediately and confusingly.

Third: thinking deltas may not arrive at all. Chapter 13 established that `display` defaults to `omitted` on Fable 5, Mythos 5, Opus 5, Sonnet 5, Opus 4.8, Opus 4.7 and Mythos Preview. Under `omitted`, **no `thinking_delta` events are sent**: the thinking block opens, receives a single `signature_delta`, and closes. The tokens were generated and you are billed for them; you simply do not see them cross the wire.

That has a direct and pleasant latency consequence, and it is the reason Chapter 13's setting belongs in a chapter about streaming. Under `summarized`, a reasoning model streams its summary first and its answer

second, so the user watches reasoning arrive before the thing they asked for. Under `omitted`, the first *text* delta is the first delta with words in it. **Time-to-first-token is unchanged; time-to-first-useful-token improves by the entire duration of the thinking summary.** The documentation says omitting reduces latency, not cost, and this is the mechanism.

17.5 Chunky Delivery, and the Stalls You Must Design For

Deltas do not arrive at a steady rate, and the documentation warns about this explicitly for tool use: *"Current models only support emitting one complete key and value property from `input` at a time. As such, when using tools, there may be delays between streaming events while the model is working."*

So a stream is not a metronome. It is bursty. A block may sit silent for a second while the model composes a whole parameter value, then deliver it in a rush. Three consequences: do not use inter-delta gaps as a health check (a five-second silence inside a tool input is normal — use `ping` events and an overall wall-clock budget); do not size buffers by delta count, because delta sizes vary by orders of magnitude; and do not render character-by-character on a fixed timer unless you deliberately decouple arrival from display, which is an honest admission that you are animating rather than reporting.

17.6 Fine-Grained Tool Streaming

By default the API **buffers and validates** each tool parameter value before streaming it back to you. That means a large parameter — a file body, a block of generated code, a long document — produces nothing at all until the model has finished generating it, and then arrives at once.

Fine-grained tool streaming turns that buffering off. Set `eager_input_streaming: true` on any user-defined tool, and enable streaming on the request:

```
{"name": "make_file",
 "description": "Write text to a file",
 "eager_input_streaming": true,
 "input_schema": {"type": "object",
                  "properties": {"filename": {"type": "string"}},
                  "required": ["filename"]}}
```

It is supported on all models, on the Claude API and on every partner platform. It is a **per-tool** field, which matters: you can stream the one tool with a large parameter and leave everything else buffered.

The legacy beta header `fine-grained-tool-streaming-2025-05-14` turned it on for every tool that left the field unset. The per-tool field replaces it, and an explicit `false` keeps buffered streaming for that tool even when a request still sends the old header. If you inherit a codebase with that header in it, the correct migration is to set the field per tool and delete the header, in that order.

The price of eagerness is stated plainly in the documentation and is the reason this is opt-in: **the API does not validate what it streams**, so the accumulated string may not be valid JSON. It can also be cut off mid-parameter by `stop_reason: "max_tokens"`.

The documented handling is precise, and it is a small masterpiece of protocol design worth copying in your own systems. When the accumulated input will not parse, you cannot run the tool — so you tell the model, in the same channel it would have received a result on:

```
{"type": "tool_result",
 "tool_use_id": "toolu_01A09q90qw90lq917835lq9",
 "is_error": true,
 "content": "{\"INVALID_JSON\": \"<the unparseable input>\"}"}
```

Wrapping the raw string under a single `INVALID_JSON` key makes the failure unambiguous to the model and preserves the original for your debugging. Build that wrapper with a JSON library, never by concatenating strings — the thing you are escaping is by definition malformed.

> TIP
>
> **When fine-grained streaming is worth it**
>
> Only when a **human is watching a large parameter appear**, or when you can begin work on a prefix of the parameter before it is complete — writing bytes to a file, for instance. If your code parses the input and then acts, eager streaming buys you nothing and costs you a validation guarantee. Chapter 34 returns to this under error contracts.

17.7 Errors Arrive Inside a 200

Chapter 16 established that a **refusal is an HTTP 200**. Streaming extends that principle: so is an overload.

```
event: error
data: {"type": "error", "error": {"type": "overloaded_error",
                                  "message": "Overloaded"}}
```

That is an `overloaded_error` — the thing that would have been an HTTP 529 on a non-streaming call — delivered in the body of a response whose status line said 200 several seconds ago. By the time the server knows it is overloaded, the headers are long gone.

The rule that follows is blunt and applies to every streaming API you will ever integrate: **once a stream has started, the HTTP status code is history.** Your error handling has to live in the event loop, not around it. Chapter 29 builds the retry and backoff policy on top of this; what matters here is that an `error` event must be a branch in your reader, not an unhandled key.

Recovery differs by model generation, and this is a genuine trap for anyone maintaining a mixed fleet:

Models	How to resume an interrupted stream
Claude 4.5 and earlier	Capture the partial text, send it back as the **start of an assistant message**, and continue.
Claude 4.6 and later	Capture the partial text, send it back inside a **user message** with an instruction to continue from where it left off.

The reason for the change is Chapter 16's: prefilling is not supported on 4.6 and later, and resuming a stream by prefixing an assistant turn *is* prefilling. Both recipes share the same caveat: **tool use and thinking blocks cannot be partially recovered.** You may resume from the most recent text block; anything else must be regenerated.

17.8 The 21,333 Rule

The SDKs refuse a **non-streaming** request whose `max_tokens` exceeds **21,333**, and require you to stream instead.† The number is not an API limit — the API will accept the request — it is a client-side guard against an HTTP read timeout, derived from a plausible worst-case decode rate over a ten-minute connection.

Chapter 11 gives you the intuition for why the guard exists and why the figure is so specific. Decode is 82–99 per cent of wall clock and proceeds one token at a time. A 128,000-token response is not a big response; it is a *long* one, measured in minutes. Non-streaming means a single socket held open for all of them, with nothing crossing it. Something in the path — a load balancer, a proxy, a cloud function timeout — will eventually decide that socket is dead.

The practical rule: **if `max_tokens` is large, stream, even if you intend to ignore every event.** Every SDK provides a way to do exactly that, accumulating internally and handing you the finished `Message`: `get_final_message()` in Python, `finalMessage()` in TypeScript, `Accumulate(event)` in Go, `MessageAccumulator` in Java, `.Aggregate()` in C#, `.accumulated_message` in Ruby. You get streaming's connection behaviour with none of streaming's client complexity, and you should reach for it by default.

17.9 Streaming Is Perception, Not Physics

Chapter 11 measured this and the conclusion is worth restating in the chapter that owns the feature, because it is routinely misunderstood by people who ought to know better.

Streaming does not make generation faster. The model produces tokens at the same rate either way. Streaming changes **when you are allowed to look**. Time-to-first-token improves enormously; total time to the last token is unchanged, or very slightly worse for the overhead of framing.

What streaming genuinely buys you is **perceived latency** — a user reading at 250 words per minute is slower than the model is writing, so once the first words appear the wait is over for them — **early cancellation**, which is the only genuine cost saving in the feature, and **connection survival** per section 17.8. What it does not buy you is throughput, cost, or time to a complete answer. If a downstream *program* is the consumer and it needs the whole answer before it can act, streaming has added complexity and removed nothing.

COMMON MISTAKE

Streaming an agent's tool-use turn

Symptom. An agent's per-step latency does not improve after a week of work adding streaming, and a new class of parse errors appears. **Cause.** The consumer is a program, not a person. It cannot act on a partial `tool_use` input, so it waits for `content_block_stop` regardless. The team added a reassembler and gained nothing but its bugs. **Fix.** Stream where a human reads. Use `get_final_message()` where a program acts. **Detection.** Ask of every stream: who is reading this before it finishes? If the answer is "nothing", you want the accumulator, not the events.

17.10 Images: Patches, Not Pixels

Now the other direction — what you send in.

Claude does not see pixels. It sees **patches**. A patch is a 28×28-pixel block of the image, and each patch is one **visual token**. So an image costs

```
visual tokens = ceil(width / 28) * ceil(height / 28)
```

and those tokens are billed at the model's ordinary **input** rate. There is no separate image price and no per-image fee. A 1000×1000 image is $\lceil 1000/28 \rceil \times \lceil 1000/28 \rceil = 36 \times 36 =$ **1,296 tokens**, which on Opus 5 at $5 per MTok is $0.00648 — about **$6.48 per thousand images**.

The formula has a sharp edge worth internalising: it is a **staircase**, not a ramp. A 28×28 image is one token. A 29×28 image is two. One extra pixel column doubles the cost of a tiny image, and — far more usefully — shaving a few pixels off a large one is usually free. Section 17.26 shows a case where the documentation's own table and this formula disagree by one pixel of width and agree exactly on the number you are billed.

The two resolution tiers

Each model has a maximum native resolution, expressed as a **long-edge limit** and a **visual-token limit**. An image over either limit is scaled down — aspect ratio preserved — to the largest size that fits both.

Tier	Models	Max long edge	Max visual tokens
High-resolution	Claude 4.7 and later	2576 px	4784
Standard	All other models	1568 px	1568

High-resolution is automatic on the listed models. There is no beta header and no client-side opt-in, which means there is also **no way to opt out except by sending a smaller image**.

That last clause is the one that costs money. A high-resolution model can charge roughly **three times** as many visual tokens for the same file as a standard-tier model: 4,784 against 1,560 for a 4K screenshot, a ratio of 3.07. If you do not need the fidelity — and for reading a bar chart or a slide you usually do not — downsample before you send. Section 17.26 recomputes the entire published table from the rule above and reproduces all twelve token counts exactly.

KEY TAKEAWAYS

A number you should be able to produce from memory

A 1000×1000 image is 1,296 tokens on every current model. That is roughly a thousand words of English. If someone proposes attaching four screenshots to every request in a chat product, you now know that is about 5,000 extra input tokens per turn before anybody has typed anything — and Chapter 16 established that per-turn input is resent every turn.

17.11 The Three Source Types

An image arrives as an `image` content block with one of three source types.

```
{"type": "image",
 "source": {"type": "base64", "media_type": "image/png",
            "data": "iVBORw0KGgoAAAANSUhEUg..."}}
```

```
{"type": "image",
 "source": {"type": "url",
            "url": "https://example.com/chart.png"}}
```

```
{"type": "image",
 "source": {"type": "file", "file_id": "file_011CPMxVD3fHLU"}}
```

Source	Payload cost	Availability	Use it when
`base64`	Image bytes × 4/3, on **every** request	Everywhere, including Bedrock and Google Cloud	One-shot, small, local.
`url`	A few dozen bytes	Claude API and most platforms	The image is already publicly hosted.
`file`	A few dozen bytes after one upload	Claude API, Claude Platform on AWS, Microsoft Foundry (Anthropic-hosted deployments)	The image recurs, or the conversation is long.

On **Amazon Bedrock and Google Cloud, only `base64` sources are currently available.** That is a portability constraint worth knowing before you design around `file_id`.

Two placement rules from the documentation, both cheap to obey:

- **Images before text.** The same principle as putting long documents before the query. Images after text still work; image-then-text works better.
- **Label multiple images.** Introduce each with a short text block — `Image 1:`, `Image 2:` — so you can refer to them by name in the prompt and in later turns. In a multi-turn conversation Claude retains every image from earlier turns, so "are these similar to the first two?" works without resending them.

17.12 The Limits, in One Place

Limit	Value
Supported media types	`image/jpeg`, `image/png`, `image/gif`, `image/webp`
Animations	Not supported; **only the first frame is used**
Max dimensions per image	8000 × 8000 px
Max images per request (200K-context models)	100
Max images per request (all other models)	600

Limit	Value
Max images per message on claude.ai	20
Max size per image	10 MB base64 (Claude API, claude.ai); 5 MB on Bedrock and Google Cloud
Max request size	32 MB on standard endpoints; lower on some partner platforms

One rule hides a real trap. **If a single request contains more than 20 images, a stricter per-image dimension limit applies.** Exceeding it produces an `invalid_request_error` whose message mentions "many-image requests" and states the current limit in pixels. On Bedrock and Google Cloud, `document` blocks such as PDFs count toward that threshold of 20 as well. The portable defence is to resize every image so that neither dimension exceeds **2000 px**, or to keep the request to 20 or fewer image and document blocks.

Note also that the 600-image ceiling is theoretical for most payloads: 600 images will hit the 32 MB request limit long before they hit the count. And even with the Files API, a request with many large images can fail before reaching 600.

17.13 What Vision Is Reliable For — Chapter 10's Triage, Applied

Chapter 10 replaced "can it?" with a triage: is this task **information**, **exact execution**, **external state**, or **checkability**? That triage is the right tool here, because "can Claude read images?" is exactly the wrong question.

Task	Triage class	Verdict
Summarise what a chart shows	Information	Reliable. This is what vision is for.
Describe a page layout, find the section headings	Information	Reliable.
Transcribe a 40-row table to CSV, exactly	**Exact execution**	Treat as unreliable. Verify every cell or use a parser.
Count 200 small objects	Exact execution	Documented as approximate. Do not.
Return bounding boxes for automation	External state + exact execution	Documented as approximate; verify before acting.
Identify a named person	—	**Refused**, by policy.
Decide whether an image is AI-generated	—	Documented as unreliable. Do not build on it.
Read a CT or MRI	—	Explicitly out of scope for the model's design.

The documented limitations also include hallucination on low-quality, rotated, or very small images — under about 200 pixels — which is the vision form of a lesson from Chapter 8: the failure mode is a confident answer, not a blank one. The architectural consequence is the one this book keeps arriving at.

Verifiability beats accuracy. If you must extract a table exactly, either parse it deterministically and use the model to interpret the result, or extract it twice and diff. Structured outputs (Chapters 10 and 23) constrain the *shape* of what comes back; they do not make a misread digit correct.

17.14 The Files API

The Files API is the create-once, use-many-times half of the story. Upload a file, get a `file_id`, and reference it instead of resending bytes.

It is **beta**, and requires the header `anthropic-beta: files-api-2025-04-14`. The SDKs add it automatically for operations under `beta.files`, but a *Messages* request that references a `file_id` must still send it.

Operation	Endpoint	Notes
Upload	`POST /v1/files`	Returns `id`, `filename`, `mime_type`, `size_bytes`, `created_at`, `downloadable`.
List	`GET /v1/files`	Paginated: `limit` (default 20), `before_id`, `after_id`.
Get metadata	`GET /v1/files/{id}`	
Download	`GET /v1/files/{id}/content`	**Only for files created by skills or the code execution tool.**
Delete	`DELETE /v1/files/{id}`	Irreversible.

The block type depends on the file type: PDFs and `text/plain` go in a `document` block; JPEG, PNG, GIF and WebP go in an `image` block; datasets destined for the code execution tool go in a `container_upload` block. Sending an image file in a document block is a documented 400.

Storage: **500 MB per file, 500 GB per organisation.** Files are scoped to the **workspace** of the API key that uploaded them — any key in that workspace can reference them, which is a fact your access model needs to account for. Files cannot be modified or renamed; to change content you upload a new file and delete the old one. They persist until deleted, and deletion cannot be undone.

All Files API operations are free. Uploading, listing, downloading, metadata and deletion cost nothing. File content used in a Messages request is priced as ordinary input tokens. During the beta, file-related calls are limited to roughly 100 requests per minute.

`downloadable` is `false` for everything you upload. Only files *created* by skills or the code execution tool can be downloaded; attempting to download your own upload is a 400. The API is not a general object store, and that asymmetry surprises people.

> WARNING
>
> **The most expensive confusion in this chapter**
>
> A `file_id` reduces the **bytes you upload**. It does not reduce the **tokens you are billed**. The image or document is expanded into exactly the same tokens either way, on every request that references it. Section 17.28 measures a ten-turn conversation about a 20-page PDF: switching from base64 to `file_id` cuts the upload from 32.0 MB to 3.2 MB and changes the bill by **exactly nothing**. The lever that changes the bill is **prompt caching**, and it is a different lever.

17.15 Documents: A PDF Is Text *and* Images

You can send a PDF as a `document` block with a `url`, `base64` or `file` source, and every currently active model supports it.

What happens on the server is the part that decides your bill:

1. Each page is **converted into an image**.
2. The text of each page is **extracted and supplied alongside** that image.
3. Claude analyses both.

So a page is billed as **text tokens plus visual tokens** — not the larger of the two, and not one instead of the other. That is what makes PDFs feel unexpectedly expensive, and it is also what makes them work: the visual channel is why Claude can answer questions about a chart, a stamp, a signature or a two-column layout that no text extractor preserves.

The documented ranges: **1,500–3,000 text tokens per page** depending on density, plus the image cost from section 17.10.

Limit	Value
Max request size	32 MB (varies by platform)
Max pages per request	**600**, or **100** when the request's context window is under 1M tokens
Format	Standard PDF, no passwords or encryption

Dense PDFs can fill the context window well before the page limit, and large ones can fail before it even with the Files API. The documented remedy is to split the document into sections and to downsample embedded images.

Because PDF support runs on the vision path, it inherits every vision limitation from section 17.13 — including the one about rotated pages. The documentation's performance guidance is unglamorous and effective: place the PDF before your text, use standard fonts, rotate pages upright, refer to the page numbers a human would see in a viewer, split large files, and enable prompt caching for repeated analysis.

Plain text files — `.txt`, `.csv`, `.md` — can go in a `document` block too, uploaded with MIME type `text/plain`. **Binary office formats cannot**: `.xlsx` and `.docx` must be converted to text or PDF first. For a `.docx` containing images, the documented route is to convert to PDF, which buys you the image parsing

and citations.

17.16 Documents and Citations

Chapter 8 introduced `citations: {"enabled": true}` and the guarantee that comes with it: the pointers a citation returns are always valid, `cited_text` costs no output tokens, and citations are **incompatible with structured outputs** (a 400). Documents are where that feature earns its keep, because a `document` block takes the flag directly:

```
{"type": "document",
 "source": {"type": "file", "file_id": "file_011CNha8iCJcU1wXNR"},
 "title": "Q3 Board Pack",
 "context": "Internal, unaudited.",
 "citations": {"enabled": true}}
```

`title` and `context` are optional and both are worth setting. The title is what a citation names; the context is a place to put provenance the model should know but that is not in the document.

One platform-specific gotcha, and it is a big one. On **Amazon Bedrock's Converse API** (the legacy path, Opus 4.6 and earlier), there are two document modes, and **citations is the switch between them**. Without citations enabled, Converse falls back to plain text extraction — roughly 1,000 tokens for a 3-page PDF, and **no ability to read charts, graphs or layout at all**. With citations enabled, you get full visual understanding at roughly 7,000 tokens for the same 3 pages. If someone reports that "Claude can't see the charts in our PDFs" on Bedrock, this is very probably why. The InvokeModel API has no such coupling.

17.17 Visual Analogies

> VISUAL ANALOGY
>
> **Streaming is a dictated letter, not a delivered one**
>
> A non-streaming call is a letter arriving in an envelope: complete, signed, dated. A streaming call is someone dictating that letter down a phone line while you write it out. You get the words sooner. You also become the one responsible for the paper, the order of the paragraphs, and for noticing if the line goes dead — and crucially, the *sign-off* comes last. Until you hear "yours sincerely", you do not know whether the letter finished or the phone died.
>
> **Where it breaks.** A dictating human corrects themselves, backtracks, and can be asked to repeat. The API does none of these. Deltas are append-only, never revised, and there is no "sorry, could you say that again?" — the recovery procedure in section 17.7 starts a *new* request, and on 4.6 and later it does so by asking the model to continue, which is not at all the same as replaying what was lost.

VISUAL ANALOGY

An image is a page of a book you have to buy by the square inch

You are not charged for what is *in* the picture; you are charged for how much picture there is. A 28x28 patch is one square inch, and you buy whole square inches — 29 pixels wide costs the same as 56.

Where it breaks. Square inches of paper are the same everywhere, whereas the tiers mean an identical file is a different number of tokens on Opus 5 and Haiku 4.5. And unlike paper, more is not always better: past the tier cap, extra resolution is discarded before the model ever sees it, so you pay to render a bigger file and Claude looks at the same picture. Section 17.27 shows 200 dpi and 300 dpi producing byte-for-byte identical bills.

VISUAL ANALOGY

The Files API is a cloakroom ticket; prompt caching is a season ticket

Handing over a `file_id` is like leaving a coat at a cloakroom and carrying a ticket. You are not lugging the coat around — the *payload* is small — but every time you go into the building, the coat is fetched and put on you again, and you pay the same entry fee.

Where it breaks. A cloakroom saves you effort; a `file_id` saves your *network* effort and nothing else, and it will not save your bill. Prompt caching is the season ticket: it changes the price of entry, at 0.1x for a read. Confusing the two is section 17.28's measured failure.

17.18 Architecture: Where the Stream Terminates

The single most consequential architectural decision about streaming is **where the stream ends**, and it is usually made by accident.

```
(a) Stream terminates at your server  -- the usual right answer

  browser <--- your protocol ---- app server ---- SSE ---> API
              (WebSocket/SSE)          |
                                  reassembler
                                  stop_reason branch
                                  policy, redaction, audit
                                  side effects gated here

(b) Stream terminates at the browser  -- looks simpler, is not

  browser <-------------- SSE (proxied) --------------> API
      |
  reassembler in JavaScript
  stop_reason branch in JavaScript
  ...and your API key is somewhere it should not be

In (a) you re-emit your own event protocol, so you can insert a
"retracted" or "refused" event that the Messages API has no way
to express. In (b) you cannot: the last thing the client hears
before a dropped connection is indistinguishable from an answer.
```

Figure 17.2 — Two placements of the same stream

Placement (a) costs you a hop and buys three things you cannot get otherwise: the API key never leaves your infrastructure; the `stop_reason` branch happens somewhere you can log and alert on it; and your own event protocol can carry states the Messages API cannot — most importantly a *retraction*. The widget in the introduction could not tell its user "that answer was incomplete" because it had no event

for it. The reassembler belongs in one adapter module, exactly as Chapter 16 argued for `parse_response`: **one reassembler, one place where a stream is declared finished, one place where side effects are permitted.**

17.19 Examples: Four Requests

A. Stream a chat reply.

```
{"model": "claude-sonnet-5", "max_tokens": 1024, "stream": true,
 "system": "You are a support agent for Acme Ltd.",
 "messages": [{"role": "user", "content": "Where is my order?"}]}
```

B. A long generation you do not want to watch. `max_tokens` is over 21,333, so streaming is not optional — but nothing reads the events; the SDK accumulates and hands back a `Message`.

```
with client.messages.stream(model="claude-opus-5",
                            max_tokens=128000,
                            messages=[{"role": "user",
                                       "content": "Write the report."}]
                            ) as stream:
    message = stream.get_final_message()
```

C. Two images and a question, images first, each labelled.

```
{"model": "claude-opus-5", "max_tokens": 1024,
 "messages": [{"role": "user", "content": [
   {"type": "text", "text": "Image 1:"},
   {"type": "image", "source": {"type": "file", "file_id": "file_a"}},
   {"type": "text", "text": "Image 2:"},
   {"type": "image", "source": {"type": "file", "file_id": "file_b"}},
   {"type": "text", "text": "Which quarter differs, and by how much?"}
 ]}]}
```

D. A cached PDF with citations on. The `cache_control` marker is what makes turns two onward cheap; the `file_id` is what makes them small.

```
{"model": "claude-opus-5", "max_tokens": 2048,
 "messages": [{"role": "user", "content": [
   {"type": "document",
    "source": {"type": "file", "file_id": "file_board_pack"},
    "title": "Q3 Board Pack",
    "citations": {"enabled": true},
    "cache_control": {"type": "ephemeral"}},
   {"type": "text", "text": "Which segment missed plan, and cite it."}
 ]}]}
```

Note that `cache_control` sits on the *document* block, not on the request. Chapter 11's misplaced-prefix failure — a cache marker placed after the variable content, costing 20.6 per cent *more* than no caching at all — is exactly as easy to commit here. The document is the stable prefix; the question is the variable suffix.

17.20 Code Walkthrough: A Parser, and the Thing It Refuses to Do

The mini project extends Chapter 16's `wire.py` rather than replacing it, because the continuity is the point: a streamed response and a non-streamed response are the same object, and the test that proves it is the centre of this chapter.

Start with the frame parser. Twenty lines, no dependencies.

```
def iter_sse(raw):
    for frame in raw.replace("\r\n", "\n").split("\n\n"):
        name, data_lines = None, []
        for line in frame.split("\n"):
            if not line or line.startswith(":"):
                continue
            field, _, value = line.partition(":")
            if value.startswith(" "):
                value = value[1:]
            if field == "event":
                name = value
            elif field == "data":
                data_lines.append(value)
        if name is None and not data_lines:
            continue
        payload = json.loads("\n".join(data_lines)) if data_lines else {}
        yield name or payload.get("type"), payload
```

Line by line. `replace("\r\n", "\n")` because SSE is specified with CRLF and real servers send both. Splitting on a blank line is the frame boundary — that blank line is the whole protocol. A line starting with `:` is a comment; some servers use them as keep-alives instead of `ping` events. `partition(":")` splits field from value, and the single leading space after the colon is stripped as the specification requires. Multiple `data:` lines are **joined with newlines**: the API sends one, but a parser that assumes one is a parser that breaks the day it sends two. The final fallback covers transports that hand you the JSON without the event name.

Now the reassembler. Three pieces of state do all the work:

```
self.message = None    # the envelope, from message_start
self.blocks = {}       # index -> content block dict
self.json_buf = {}     # index -> accumulated partial JSON string
```

Every one of those dictionaries is **keyed by** `index`, and that is the single design decision that separates this from the accumulator in section 17.24. Applying a delta is then a dispatch on `delta["type"]`:

```
if kind == "text_delta":
    block["text"] = block.get("text", "") + delta["text"]
elif kind == "thinking_delta":
    block["thinking"] = block.get("thinking", "") + delta["thinking"]
elif kind == "signature_delta":
    block["signature"] = block.get("signature", "") + delta["signature"]
elif kind == "input_json_delta":
    self.json_buf[i] = self.json_buf.get(i, "") + delta["partial_json"]
else:
    self.unknown.append(("content_block_delta", kind))
```

The `else` branch is not defensive padding. The versioning policy states that new event and delta types may be added and that your code should handle unknown types gracefully. Recording them is graceful; crashing is not; **silently dropping them is the worst of the three**, because the day a new delta type carries something you needed, nothing will tell you.

`content_block_stop` is where a tool input becomes an object — `json.loads` on the accumulated buffer, wrapped in a `try/except json.JSONDecodeError` that records the raw string instead of raising, because section 17.6 says fine-grained streaming does not validate. An unparseable input is a `tool_result` with `is_error: true`, not a stack trace.

`message_delta` merges top-level changes and **replaces** `usage` rather than adding to it — the counts there are cumulative, and getting that wrong double-counts every output token in your billing dashboard.

And then the part that matters most, which is a refusal:

```
def payload(self, strict=True):
    if self.error is not None:
        raise StreamError(self.error["message"])
    if self.message is None:
        raise IncompleteStream("no message_start received")
    if strict and not self.complete:
        raise IncompleteStream(...)
```

where `complete` is `self.done and not self.open_indices` — `message_stop` seen **and** every block closed. In strict mode a stream that ended early does not produce a message. It produces an exception. `strict=False` exists so you can *render* a partial, but you have to ask for it, by name, at the call site. That one keyword argument is the entire difference between this module and the widget in the introduction.

17.21 Mini Project: An SSE Reassembler and a Price List for Pictures

What you are building. Two things that share a rate card: a from-scratch streaming client that can prove it produces the same object as the non-streaming one, and a token-and-cost model for images and documents that can prove it matches the published table.

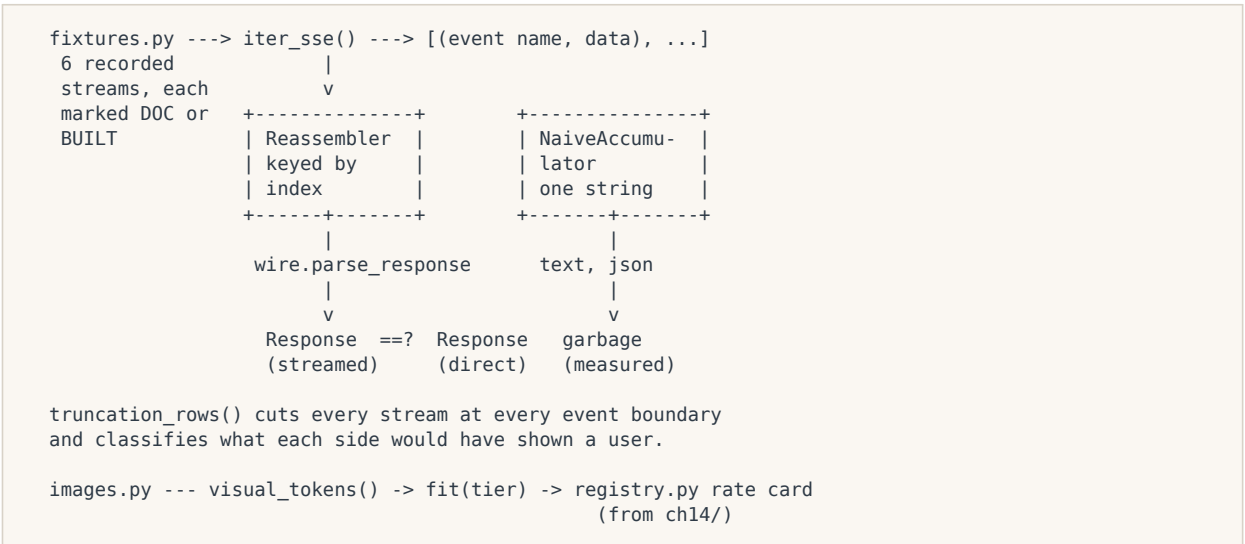


Figure 17.3 — ch17/ architecture

Folder structure.

```
ch17/
  wire.py         copied from ch16/  -- unchanged
  registry.py     copied from ch14/  -- unchanged
  fixtures.py     six SSE streams + the non-streaming equivalents
  sse.py          iter_sse, Reassembler, NaiveAccumulator, harness
  images.py       visual tokens, tiers, PDF and conversation costing
  report.py       every number in this chapter
  test_sse.py     28 tests
  test_images.py  26 tests
```

`wire.py` and `registry.py` are copied, not imported across directories, exactly as `ch15/` and `ch16/` did. Keep them in sync.

Provenance, again. Chapter 14 established that a fact is verified on a date or it is UNKNOWN. `fixtures.py` extends the rule to fixtures. Every stream declares itself either **DOC** — transcribed from `platform.claude.com/docs`, 30 July 2026 — or **BUILT** — constructed here to exercise a contract the documentation states but does not print a full stream for. Four are DOC. Two are BUILT: the parallel-tool stream, which exercises the documented index-keyed accumulation contract, and the refusal stream. A test asserts that every fixture declares a provenance and a note. Nothing in this chapter is presented as a recording of a call that was not made.

Run it.

```
cd ch17
python3 report.py
python3 -m pytest -q      # 54 passed
```

17.22 Panel A — One Frame, and the Field That Is Not In It

```
event name : message_start
data type  : message_start
stop_reason in message_start : None
full event sequence:
   1. message_start
   2. content_block_start
   3. ping
   4. content_block_delta
   5. content_block_delta
   6. content_block_stop
   7. message_delta
   8. message_stop
```

Eight events to say "Hello!". The `ping` is real and is in the documentation's own example, sitting between the block opening and its first delta — which is a useful reminder that keep-alives arrive at moments you would not have chosen.

`stop_reason` is `None` in event 1 and is set in event 7. Between them sit the two events that carry the answer. **The user can read the whole reply two events before your program is allowed to have an**

opinion about it.

17.23 Panel B — The Headline: Reassembly Equals the Direct Call

```
fixture             src    events blocks    stop_reason  equal?
BASIC               DOC         8      1        end_turn  True
TOOL_USE            DOC        27      2        tool_use  True
THINKING            DOC        13      2        end_turn  True
WEB_SEARCH          DOC        25      4        end_turn  True
PARALLEL_TOOLS      BUILT      11      2        tool_use  True
REFUSAL             BUILT       3      0         refusal  True
```

Six streams, six exact matches. `Response` is a dataclass, so `==` compares `id`, `model`, `content`, `stop_reason`, `stop_sequence`, `stop_details` and `usage` field by field. The right-hand side of each comparison is **hand-written** in `fixtures.NONSTREAM` from the documented block shapes — not generated by the reassembler, because a test that compares a program against its own output proves nothing.

Read the last row twice. A refusal streams as three events and produces a message with **zero content blocks**, `stop_reason: "refusal"` and `stop_details.category`. A renderer that shows text as it arrives shows nothing at all and then must explain itself. Chapter 16 made the point that a refusal is an HTTP 200; the streaming form is stranger still, because for two of its three events it is indistinguishable from a response that is simply thinking.

And the second row: `TOOL_USE` reassembles to `stop_reason: "tool_use"`, which `wire.py` reports as **non-terminal**. The stream finished. The turn did not.

17.24 Panel C — The Naive Accumulator, Failing Twice

```
PARALLEL_TOOLS, ignoring `index`:
  accumulated : {"city":{"city": "Paris"} "London"}
  json.loads  : JSONDecodeError - Expecting ',' delimiter: line 1 ...
  keyed by index: [{'city': 'Paris'}, {'city': 'London'}]
  -> this one fails LOUDLY. You will find it in an hour.
WEB_SEARCH, ignoring `index`:
  naive  : "I'll check the current weather in New York City for
             you.Here's the current weather information for New York
             City: 18 C and overcast."
  blocks :
    [0] "I'll check the current weather in New York City for you."
    [3] "Here's the current weather information for New York City:
         18 C and overcast."
  -> this one fails SILENTLY. Two blocks became one sentence, and
     the join is inside a full stop. Nothing raises.
```

Two failures from one omitted dictionary key, and they are not equally dangerous.

The first is the good kind. Two tool inputs interleave into `{"city":{"city": "Paris"} "London"}`, `json.loads` raises, and somebody fixes it before lunch.

The second is the kind that ships. The response genuinely has four content blocks — text, a server tool use, its result, and more text. Concatenating every `text_delta` yields a string that is grammatical, plausible, and wrong: `for you.Here's` has lost a block boundary, and with it the fact that a web search happened in between. Nothing raises. No test that checks "did we get text?" will fail. The output is subtly, silently reordered relative to reality — the same species of failure as Chapter 5's lexical retrieval mis-ranking and Chapter 8's grounding check passing a fabrication.

KEY TAKEAWAYS

The rule this panel exists to establish

`index` is not metadata. It is the only thing that distinguishes two content blocks from one. Any buffer that accumulates deltas must be keyed by it — which is precisely why the documentation's own accumulation example is a `dict[int, str]` and not a `str`.

17.25 Panel D — Ninety-Three Cut Points

Every stream is cut at every event boundary, and each prefix is handed to both readers.

```
fixture          cuts       EMPTY     TRUNCATED  BLOCKS_GLUED    UNPROVEN    COMPLETE
BASIC               9           4              1             0           3           1
TOOL_USE           28           4             12             0          11           1
THINKING           14          10              0             0           3           1
WEB_SEARCH         26           3             16             6           0           1
PARALLEL_TOOLS     12          11              0             0           0           1
REFUSAL             4           3              0             0           0           1
TOTAL              93          35             29             6          17           6
strict reassembler raises on 87 of 93 prefixes.
naive accumulator raises on 0 of 93.
52 prefixes hand a user non-empty text with no signal:
  29 truncated, 6 with blocks glued, 17 complete but unproven.
stop_reason is known at only 12 of 93 cut points (12.9%).
```

The classes are defined by what a *user* would see:

- **EMPTY** — nothing rendered yet. No harm done. 35 of 93.
- **TRUNCATED** — a proper prefix of the answer, presented as if it were the answer. This is the introduction's incident. 29.
- **BLOCKS_GLUED** — two or more text blocks concatenated, so a boundary the model put there has been erased. 6, all in `WEB_SEARCH`, because it is the only fixture with two text blocks.
- **UNPROVEN** — every character of the answer, but no `stop_reason`, so `end_turn` and `max_tokens` are indistinguishable. 17. This is the quietest class and in some ways the most interesting: the text is *right*, and you still cannot ship it, because "the model finished" and "the connection finished" are different claims.
- **COMPLETE** — 6, one per stream, and only at the very end.

Three numbers deserve to be read slowly. **87 against 0**: the strict reassembler correctly refuses 87 prefixes and the naive one refuses none — not because it is optimistic, but because refusing is not an operation it has. **52 of 93**: more than half of all possible interruption points produce a screenful of confident text with nothing to mark it, which is much worse than intuition suggests because intuition

imagines the drop happening early. **12 of 93**: for 87 per cent of these events, `stop_reason` is unknown. Chapter 16 called it the control-flow branch; streaming's contribution is that the branch is unavailable for almost the entire time your program is doing things.

17.26 Panel E — The Official Table, Recomputed

```
        sent |          standard tier |          high-res tier
             |           seen     tok |           seen     tok
  200x200    |    200x200          64 |    200x200          64
 1000x1000   |   1000x1000       1296 |   1000x1000       1296
 1092x1092   |   1092x1092       1521 |   1092x1092       1521
 1920x1080   |   1456x819        1560 |   1920x1080       2691
 2000x1500   |   1270x952        1564 |   2000x1500       3888
 3840x2160   |   1456x819        1560 |   2576x1449       4784
All twelve token counts match the published table exactly.
One dimension differs: the table prints 1269x952 where the
rule as stated yields 1270x952. Both are 46x34 patches, so
both are 1564 tokens. The staircase makes the pixel free.
and the documented dollar examples, recomputed:
  1000x1000 on claude-haiku-4-5      1296 tok  $  1.30 per 1,000 images
  1000x1000 on claude-opus-5         1296 tok  $  6.48 per 1,000 images
  3840x2160 on claude-opus-5         4784 tok  $ 23.92 per 1,000 images
```

Twelve token counts and three dollar figures, all reproduced from `ceil(w/28) x ceil(h/28)` plus the tier limits and Chapter 14's rate card. Nothing here is memorised; all of it is derived, which means you can derive it for *your* image sizes rather than looking for your case in somebody's table.

The one disagreement is reported rather than hidden, in keeping with Chapter 12's discipline: the rule as stated admits 1270x952 where the table prints 1269x952. Both are 46 x 34 patches and both cost 1,564 tokens, and `test_images.py` asserts the discrepancy explicitly rather than papering over it with a tolerance.

Note rows four and six together: **a 3840x2160 image costs no more than a 1920x1080 one on the standard tier** — 1,560 tokens for both, because both are downscaled to the identical 1456x819. Sending a 4K screenshot to a standard-tier model is not expensive. It is merely pointless.

17.27 Panel F — A Twenty-Page PDF, Priced

```
  dpi      page px         seen  vis/pg   tokens   img %      USD
   72   595x842      595x842        682    53640   25.4%   0.2682
  100   827x1169     827x1169      1260    65200   38.7%   0.3260
  150  1240x1754    1240x1754      2835    96700   58.6%   0.4835
  200  1654x2338    1624x2296      4756   135120   70.4%   0.6756
  300  2481x3507    1624x2296      4756   135120   70.4%   0.6756
150 dpi, Opus 5: 96,700 input tokens, $0.4835.
The same content as plain text: $0.2000.
The pictures cost you 2.42x.
Rendering at 100 dpi instead: 65,200 tokens, $0.3260 (32.6% cheaper).
The same PDF on Haiku 4.5 (standard tier): page seen at 924x1307,
1551 visual tokens per page, 71,020 total.
```

Twenty A4 pages, 2,000 text tokens per page (the middle of the documented 1,500–3,000 range), Opus 5 at $5 per MTok.

A PDF costs 2.42× what the same words cost as text. That is the headline, and it is not a criticism of PDF support — the 58.6 per cent of tokens that are pictures are exactly what lets Claude answer a question about a chart. It is an argument for *knowing which you need*. If your document is a plain contract and you have reliable text extraction, sending the text is 20p where the PDF is 48p, and at ten thousand documents a month that is £2,800 a year for pictures of paragraphs.

Above the tier cap, resolution is free and worthless. 200 dpi and 300 dpi produce byte-for-byte identical bills — 135,120 tokens each — because both are downscaled to 1624×2296, the largest A4-shaped page that fits 4,784 visual tokens. Rendering a PDF at 300 dpi to "give the model more to work with" costs you upload bandwidth and gives the model precisely the same picture. Meanwhile, dropping from 150 to 100 dpi takes a third off the bill.

And the token count is a property of the (document, model) pair. The identical file is 2,835 visual tokens per page on Opus 5 and 1,551 on Haiku 4.5, because they sit in different tiers. Chapter 15 chose a model on capability, speed, cost and effort. Add this: **on a vision workload, changing model changes the token count of your input, not just its price.** A cost model that multiplies a fixed token count by a per-model rate is wrong for images, and wrong in the direction that flatters the expensive model.

17.28 Panel G — The Files API Does Not Do What You Think

```
turns   no cache     cached    saved
    1     0.4913     0.6122   -24.6%
    2     0.9844     0.6685    32.1%
    5     2.4745     0.8386    66.1%
   10     4.9940     1.1256    77.5%
   20    10.1680     1.7132    83.2%
A 2.4 MB PDF base64-encodes to 3.2 MB.
Ten turns, base64: 32.0 MB uploaded.
Ten turns, file_id: 3.2 MB uploaded, once.
Token cost of both: $4.9940. IDENTICAL.
```

This is the panel to show a team that has just spent a sprint migrating to the Files API to reduce their bill.

The migration is worth doing. It cuts the bytes on the wire tenfold, keeps every request comfortably under the 32 MB limit, and removes a class of timeout. Chapter 11's finding that prefill is a real and growing share of latency applies: shipping 3.2 MB of base64 on every turn is not free in wall-clock terms.

But the token count does not move by one. The document is expanded into the same 96,700 tokens whether it arrived as base64 or as a `file_id`, on every single turn, because the Messages API is stateless and turn *n* carries the document again.

The lever that moves the bill is **prompt caching**, and it moves it a long way — 77.5 per cent by turn ten, 83.2 per cent by turn twenty. Note turn one: **caching is 24.6 per cent dearer**, because a cache write is billed at 1.25×. That is the same shape as Chapter 16's test asserting that turn one costs 25 per cent more, and it is the reason a benchmark that measures a single request concludes, wrongly, that caching is a tax.

KEY TAKEAWAYS

Two levers, two problems

`file_id` is a **bandwidth** optimisation: it changes what you upload. `cache_control` is a **token** optimisation: it changes what you are billed. They are complementary and neither substitutes for the other. Use both on a document you will ask more than one question about — Example D in section 17.19 does exactly that.

17.29 Evaluation and Deployment

What "correct" means for this module, and how you would know:

Fifty-four tests cover it: five on frame parsing (comments, CRLF, multi-line `data`, name fallback), six parametrised comparisons of reassembly against hand-written non-streaming payloads, assertions that `stop_reason` is null in `message_start` and that `usage` is cumulative rather than additive, two proving unknown deltas and unknown event names are non-fatal, one pinning all seven truncation figures, twelve parametrised assertions against the published token table, and three against the documented dollar examples.

Deployment guidance, if you were to ship a reassembler rather than an SDK — and you should ship an SDK:

- Put the reassembler behind **one** interface with `strict` defaulting to `True`.
- Emit a metric per stream: `completed`, `truncated`, `errored`, plus the `stop_reason` histogram. The introduction's incident was invisible because none of these existed.
- Alarm on `unknown` deltas and events. A new type appearing is news.
- Keep the fixtures. When the API adds an event type, a new fixture is the cheapest possible regression test.

17.30 Hands-on Labs

Lab 1 — Parse a frame by hand. Write `iter_sse` from the specification without looking at section 17.20. Feed it `fixtures.BASIC`. *Success criterion:* eight events, in the documented order, with `ping` at position three.

Lab 2 — Find the moment the answer becomes readable. For `fixtures.TOOL_USE`, print the cut index at which the naive accumulator first holds the complete text, and the cut index at which `stop_reason` becomes known. *Success criterion:* you can state the gap in events, and explain what a user is looking at during it.

Lab 3 — Break the accumulator on purpose. Modify `NaiveAccumulator` to key by `index` and rerun `test_sse.py`. *Success criterion:* the two failure tests in section 17.24 now fail — and you can say why that is a *good* outcome for a test suite whose job is to pin a bug in place.

Lab 4 — Price your own screenshot. Take a real screenshot, note its dimensions, and compute its cost on Opus 5 and on Haiku 4.5 using `images.image_cost`. Then resize it so the long edge is 1568 px and recompute. *Success criterion:* a number for the saving, and a judgement about whether the smaller image

is still legible.

Lab 5 — Decide text or PDF. Take a real 10-page PDF. Estimate its token cost with `images.pdf_cost`, then extract its text and count those tokens separately. *Success criterion:* a defensible one-sentence recommendation for that document, naming what you would lose by sending text alone.

17.31 Lab Solutions

Lab 1. The specification is in section 17.20. The two things people miss are the comment line and the blank-line frame boundary; splitting on `"\n"` instead of `"\n\n"` yields eight *lines* per frame rather than one frame.

Lab 2. The naive accumulator holds the complete text after cut 15 — the `content_block_stop` for block 0. `stop_reason` becomes known at cut 27. That is a gap of **twelve events**, all of them the tool-input deltas, during which a user is looking at a finished-looking sentence ending in a colon and your program does not yet know the turn will end in `tool_use` rather than `end_turn`.

Lab 3. `test_naive_accumulator_glues_two_text_blocks_into_one` and `test_naive_accumulator_corrupts_parallel_tool_input` both fail. This is the point Chapter 16 made about keeping the bug as a named function: a test that asserts a *defect* is documentation with a build failure attached. When someone "fixes" the naive accumulator, the suite tells them they have removed the demonstration, not found a bug.

Lab 4. A 2560×1440 screenshot is 92 × 52 = 4,784 tokens on Opus 5 — the tier cap exactly — and 1,540 tokens on Haiku 4.5. Resized so the long edge is 1568, it becomes 1568×882 → 56 × 32 = 1,792 tokens on Opus 5, a **62.5 per cent** reduction. Whether it is still legible depends entirely on the smallest text in it, which is why the documentation tells you to inspect the actual image you send.

Lab 5. The arithmetic is section 17.27's. The judgement is not arithmetic: send text alone only if you can say what is in the document that is *not* in the text. Charts, signatures, stamps, tick-boxes, handwriting, two-column layouts and tables whose meaning depends on alignment are all invisible to a text extractor and all visible to the vision path.

17.32 Best Practices

BEST PRACTICE

Fourteen habits

1. **Default to `get_final_message()`.** Stream for the connection, not for the events, unless a human is reading.
2. **One reassembler.** If `content_block_delta` appears in two files, you have two.
3. **Key every buffer by `index`.** No exceptions, ever, for any delta type.
4. **Never act on a stream that has not reached `message_stop`.** Render, yes. Act, no.
5. **Branch on `stop_reason` after the stream, exactly as in Chapter 16.** Streaming changes when it arrives, not what it means.
6. **Handle the `error` event.** A 200 is not a promise.
7. **Treat unknown event and delta types as news**: record, alert, continue.
8. **Terminate the stream at your server**, and re-emit your own protocol with a retraction event in it.
9. **Stream above `max_tokens` 21,333**, always.
10. **Put images before text**, and label them when there is more than one.
11. **Resize before you upload.** Long edge 1568 for standard-tier work, 2000 px if a request will carry more than 20 images.
12. **Use `file_id` for anything referenced more than once**, and know that it is a bandwidth fix.
13. **Cache the document, not the question.** The `cache_control` marker goes at the end of the stable prefix.
14. **Estimate before you send.** `count_tokens` is free (Chapter 6) and takes images and documents.

17.33 Common Mistakes

COMMON MISTAKE

Rendering deltas and calling it done

Symptom. Occasional truncated answers in production that nobody can reproduce; no errors in the logs. **Cause.** The client concatenates `text_delta`s and never checks for `message_stop`. Section 17.25: 52 of 93 cut points produce exactly this. **Fix.** Strict reassembly by default; `strict=False` only where a partial render is intended. **Detection.** Emit a `stream_completed` counter alongside `stream_started`. If they differ, you have found the incident.

COMMON MISTAKE

Accumulating without `index`

Symptom. Either a `JSONDecodeError` on parallel tool calls, or — worse — two text blocks silently glued together. **Cause.** A single string buffer instead of a dictionary keyed by block index. **Fix.** `dict[int, str]`, as the documented accumulation contract shows. **Detection.** A test with two content blocks of the same type. One fixture finds it instantly.

COMMON MISTAKE

Adding up the `usage` in every `message_delta`

Symptom. Billing dashboard reports more output tokens than the invoice. **Cause.** The counts in `message_delta` are **cumulative**, not incremental. **Fix.** Replace, do not add. **Detection.** Reconcile one day's computed tokens against Console → Usage → Export (Chapter 14).

COMMON MISTAKE

Assuming `content_block_start` gives you the tool input

Symptom. Tools are called with empty arguments. **Cause.** `content_block_start` carries `"input": {}` — a placeholder. The value arrives as `input_json_delta` strings. **Fix.** Accumulate and parse at `content_block_stop`. **Detection.** Assert that no tool is ever invoked with an input equal to `{}` unless its schema has no required properties.

COMMON MISTAKE

Treating a `ping` or an unknown event as a protocol violation

Symptom. A client that worked in staging fails the week a new feature ships. **Cause.** An exhaustive `match` with no default branch. **Fix.** Record unknown types and continue; the versioning policy says they will appear. **Detection.** Replay a fixture with an invented event name spliced in. `test_sse.py` does exactly this.

COMMON MISTAKE

Migrating to the Files API to cut the bill

Symptom. A sprint of work; bandwidth drops tenfold; the invoice does not move. **Cause.** `file_id` changes bytes, not tokens. Section 17.28. **Fix.** Add `cache_control` to the document block. 77.5 per cent by turn ten. **Detection.** Before any optimisation, write down which of *bytes*, *tokens*, *latency* it is supposed to change, and how you will measure that one.

COMMON MISTAKE

Rendering a PDF at 300 dpi for accuracy

Symptom. Large uploads, occasional 32 MB request failures, no accuracy improvement. **Cause.** The page is downscaled to the tier cap before the model sees it. 200 dpi and 300 dpi produce identical token counts and identical pictures. **Fix.** Render at the resolution the tier can actually use; 150 dpi is a good default and 100 dpi is a third cheaper. **Detection.** Compute `fit()` for your page size at your dpi. If `seen` is smaller than `page px`, you are paying to throw pixels away.

COMMON MISTAKE

Trusting an exact transcription from an image

Symptom. One wrong digit in a financial table, discovered by a customer. **Cause.** Exact transcription is *exact execution* in Chapter 10's triage, not information retrieval. **Fix.** Parse deterministically and use the model to interpret; or extract twice and diff; or check a total. **Detection.** Seed your evaluation set with tables whose columns sum to a known figure.

17.34 Performance Considerations

Streaming's performance story is entirely about *perception* and *survival*; Chapter 11's two-term model is unchanged, with decode dominating.

- **Time to first token** improves dramatically. **Time to last token** does not improve at all.
- `display: "omitted"` (Chapter 13) improves time-to-first-**text**-token by the whole duration of the thinking summary, because no `thinking_delta` events are sent. It reduces latency, not cost — you are still billed for the thinking tokens.
- **Cancellation is the only genuine cost saving** streaming offers: abandon early and you stop paying for output.
- On the input side, the largest lever in this chapter is **resolution**. Dropping a 20-page PDF from 150 to 100 dpi removes 31,500 input tokens from every request that carries it — and input tokens are prefill, which is the term you can cache.

17.35 Security Considerations

- **Never terminate the stream in the browser.** Placement (b) in Figure 17.2 puts your API key where a user can read it, and puts the `stop_reason` branch where an attacker can skip it.
- **An image is untrusted input.** Text inside a picture is still text a model will read, and prompt injection through a screenshot or a PDF page is a real technique. Chapter 58 covers the defence in depth; the rule here is that a document supplied by a user is user input, not context.
- `url` **sources fetch on Anthropic's side.** Do not point them at internal hosts, and treat a user-supplied URL as a request to fetch an arbitrary resource.
- **Files are workspace-scoped.** Any API key in the same workspace can reference an uploaded `file_id`. If you serve multiple tenants, do not share a workspace.
- **Deletion is not instantaneous everywhere.** Files become inaccessible shortly after deletion but may persist in in-flight Messages calls and associated tool uses; retention then follows the published policy. Zero-data-retention eligibility differs by feature and is documented separately.
- **Redact before you upload, not after.** A `file_id` is a durable object; a base64 blob in a request body is not. That is a point in favour of base64 for genuinely sensitive one-shot content.
- **Partial renders are a disclosure channel.** A stream cut mid-refusal may have already shown content that the refusal was about to withhold. Gate on `stop_reason` before persisting anything.

17.36 Real Production Examples

A legal review tool. 400-page bundles, well past the 100-page limit for sub-1M-context requests and past 600 in some matters. The team splits by section, uploads each section once to the Files API, enables citations, and caches the section for the duration of a review session. Splitting was forced by a limit; caching was what made it affordable; citations were what made the output defensible.

An expenses app. Photographs of receipts, taken by hand. The naive build sent 12-megapixel originals and paid the tier cap on every one. Resizing to a 1568-px long edge before upload cut visual tokens by

roughly 60 per cent with no measurable change in extraction accuracy — because a receipt's text was legible at that size, which the team confirmed by *looking at* the resized images rather than assuming.

A coding agent with a streamed diff. Uses `eager_input_streaming` on exactly one tool — the one whose parameter is a file body — and buffered streaming everywhere else. The user watches the file appear; every other tool call arrives validated.

A support widget, rebuilt. The one from the introduction. It now terminates the stream at the server, re-emits its own protocol, and has a `retracted` event. When a stream fails to complete, the partial answer greys out and a line appears: *"That answer was cut off. Ask again?"* Nothing about the model changed.

17.37 Certification Notes

Concept	Exam domain	Why it is examinable
The event sequence and `index`	D4 Prompt Engineering & Structured Output	Reassembly is a structured-output problem in disguise.
`stop_reason` in `message_delta`	D5 Context Management & Reliability	Control flow under partial information.
Gating side effects on `message_stop`	D1 Agentic Architecture & Orchestration	The proposes/disposes boundary.
`error` events inside a 200	D5	Failure handling that HTTP status codes cannot express.
Fine-grained tool streaming, `eager_input_streaming`	D2 Tool Design & MCP Integration	Trade-off: latency against validation.
Invalid tool JSON → `is_error` tool_result	D2	The documented error contract.
Visual tokens = patches	D5	Context budget on multimodal input.
Resolution tiers per model	D1	Model choice changes token counts, not just price.
Three image source types	D1	Portability: Bedrock and Google Cloud are base64 only.
Files API scope and lifecycle	D1	Workspace scoping is an access-control fact.
PDF = text + images	D5	The most commonly mis-estimated cost in the syllabus.
Citations on document blocks	D5	Grounding and verifiability.

17.38 Twenty Practice Questions

1. In which event does `stop_reason` first carry a non-null value?
2. What is the value of `stop_reason` in `message_start`?

3. Name the four `content_block_delta` delta types.
4. What does `content_block_start` carry as a `tool_use` block's `input`?
5. Why must delta buffers be keyed by `index`?
6. Are the token counts in `message_delta.usage` cumulative or incremental?
7. What arrives immediately before `content_block_stop` on a thinking block?
8. Under `display: "omitted"`, which events does a thinking block emit?
9. Which field enables fine-grained tool streaming, and at what scope?
10. What guarantee do you lose with fine-grained tool streaming?
11. How should an unparseable tool input be returned to Claude?
12. What HTTP status accompanies an `overloaded_error` in a stream?
13. How do you resume an interrupted stream on Claude 4.6 and later?
14. Above what `max_tokens` do the SDKs require streaming?
15. Give the formula for an image's visual-token cost.
16. What are the long-edge and visual-token limits of the two tiers?
17. Which four image media types are supported?
18. What is the maximum number of pages in one PDF request, and when is it 100?
19. What does a `file_id` reduce, and what does it not reduce?
20. On Bedrock's Converse API, what does enabling citations change about PDF processing?

17.39 Answers and Explanations

1. `message_delta`, the second-to-last event. *Not* `message_stop` — that carries nothing. *Not* `content_block_stop` — that ends a block, not the message.
2. `null`**.** The field is present, which is why a client that checks `"stop_reason" in message` rather than its value is wrong.
3. `text_delta`**,** `thinking_delta`**,** `signature_delta`**,** `input_json_delta`**.** A common wrong answer adds `citation_delta`; citations arrive within the block structure, not as a fifth delta type here.
4. **An empty object,** `{}`**.** A placeholder marking the slot. Answering "the full input" is the mistake in section 17.33; answering "a string" confuses it with `partial_json`.
5. **Because** `index` **is the block's position in the final** `content` **array and the only thing distinguishing concurrent blocks.** Without it, parallel tool inputs interleave and separate text blocks glue together.
6. **Cumulative.** Adding them double-counts.
7. **A** `signature_delta`**.** It verifies the thinking block's integrity. Answering "nothing" is right only if you forget that thinking blocks are signed.
8. `content_block_start`**, a single** `signature_delta`**,** `content_block_stop`**.** No `thinking_delta` at all. You are still billed for the thinking tokens.
9. `eager_input_streaming: true`**, per tool.** Not per request, and not a beta header any more — the legacy `fine-grained-tool-streaming-2025-05-14` header is superseded by the field.

10. **Server-side buffering and JSON validation.** The accumulated string may be invalid or incomplete JSON.
11. **As a `tool_result` with `is_error: true`, whose `content` is a JSON object with a single `INVALID_JSON` key** wrapping the raw string, built with a JSON library.
12. **200.** The headers were sent before the server knew. Answering 529 is right for the non-streaming case and wrong here.
13. **Send the partial text back inside a *user* message with an instruction to continue.** On 4.5 and earlier you put it in an assistant message; that is prefilling, which 4.6 and later reject.
14. **21,333.** It is an SDK guard against HTTP timeouts, not an API limit.
15. `ceil(width / 28) × ceil(height / 28)`**.**
16. **High-resolution (Claude 4.7 and later): 2576 px, 4784 tokens. Standard: 1568 px, 1568 tokens.**
17. **JPEG, PNG, GIF, WebP.** Animations are not supported; only the first frame is read.
18. **600 pages; 100 when the request's context window is under 1M tokens.**
19. **It reduces the bytes uploaded. It does not reduce the tokens billed.** Prompt caching reduces the tokens.
20. **It switches PDF processing from text extraction only to full visual understanding** — roughly 1,000 tokens for 3 pages against roughly 7,000, and the difference between not seeing charts and seeing them.

17.40 Ten Practical Exercises

1. Add a `content_block_delta` type of your own invention to a fixture; prove the reassembler records it and still completes.
2. Extend `truncation_rows` to report, per fixture, the earliest cut at which the naive result is a complete sentence. Argue why that number is the dangerous one.
3. Write a renderer that consumes `Reassembler.partial_text()` and draws a visible boundary between blocks. Run it on `WEB_SEARCH`.
4. Implement the `fallback` content block case: a `content_block_start`/`content_block_stop` pair with no deltas. Confirm nothing breaks.
5. Add `pause_turn` to the fixtures as a streamed response and assert it is non-terminal.
6. Compute the break-even number of turns at which caching a 20-page PDF becomes cheaper than not caching. Show your working.
7. Extend `images.py` to take a list of images and warn when a request exceeds 20 blocks or 32 MB.
8. Price a 100-page PDF at every dpi in Panel F and find the dpi at which it no longer fits a 1M context window.
9. Add a `count_tokens` cross-check function that would reconcile your estimate against the free endpoint, and state what you would do if they disagreed by 5 per cent.
10. Write the metric emission described in section 17.29 and define the alert threshold you would page on.

17.41 Five Challenge Labs

1. **A retracting client.** Build the server-side re-emitter from Figure 17.2(a), with `token`, `retract` and `final` events, and demonstrate a retraction on a truncated stream.
2. **A streaming budget guard.** Cancel a stream once accumulated output tokens exceed a threshold, and report the money saved against letting it run to `max_tokens`.
3. **A vision cost linter.** Given a directory of images, report the total token cost on two models, flag anything over the tier cap, and emit the resize commands that would minimise cost without dropping below a stated legibility floor.
4. **A document router.** Given a PDF, decide automatically between sending text, sending the PDF, and sending selected pages as images — and defend the decision with a number.
5. **A differential extractor.** Extract a table from a page twice, once through the vision path and once through a deterministic parser, and build the diff report that decides which cells a human must check.

17.42 Ten Interview Questions

1. Streaming makes responses faster. Argue against that sentence.
2. A colleague proposes rendering text as it arrives in a medical dosing assistant. What do you say?
3. Why is the tool input an object at `content_block_start`, a string in the deltas, and an object again at the end?
4. Where would you put the reassembler in a three-tier web application, and why not one tier lower?
5. Your invoice shows twice the output tokens your dashboard does. Name the likeliest cause.
6. When is fine-grained tool streaming a mistake?
7. A 4K screenshot on Haiku 4.5 costs fewer tokens than a 2-megapixel one on Opus 5. Explain.
8. A team migrated to the Files API and the bill did not move. What did they want, and what should they have done?
9. When should a PDF be sent as text instead, and what exactly do you lose?
10. A stream ends with `stop_reason: "refusal"` after 400 characters of text have already been shown. What does your system do next?

17.43 Summary

Streaming does not change the Messages API. It changes who assembles the answer. Set `stream: true` and the server stops handing you a message and starts handing you instructions for building one: `message_start` opens an empty envelope, a sequence of `content_block_start` / `content_block_delta` / `content_block_stop` groups fills it in, `message_delta` supplies the top-level fields, and `message_stop` closes it. Every block event carries an `index`, and that index is not decoration — it is the block's position in the final `content` array and the only thing that distinguishes two concurrent blocks. Buffers keyed by anything else produce, at best, a `JSONDecodeError` on parallel tool calls and, at worst, two text blocks glued into one grammatical, plausible, wrong sentence.

The field that decides what you do arrives second from last. `stop_reason` is `null` in `message_start` and is set in `message_delta`, which means every character you have already rendered was rendered on the

assumption that the message would end well. Cut six recorded streams at every one of their 93 event boundaries and the strict reassembler refuses 87 of them while a naive accumulator refuses none; 52 of those prefixes hand a user non-empty text with no signal whatsoever, and `stop_reason` is known at only 12 of the 93. The discipline that follows is one sentence: render if you like, but never *act* on a stream that has not reached `message_stop` with a stop reason you have explicitly handled. Errors compound the point, since an `overloaded_error` arrives inside a response whose HTTP status said 200 several seconds earlier — once a stream has started, the status code is history.

On the input side, the arithmetic is unusually clean and unusually often ignored. Claude sees 28×28 patches, so an image costs `ceil(w/28) × ceil(h/28)` visual tokens at the ordinary input rate, capped by a per-model tier: 2576 px and 4,784 tokens on Claude 4.7 and later, 1568 px and 1,568 tokens on everything else. That rule reproduces all twelve token counts and all three dollar figures in the official table exactly, which means you can compute your own case rather than look for it in someone's table. It also produces two facts worth carrying: high-resolution models can charge three times as much for the same file, and past the tier cap extra resolution is discarded before the model sees it.

Documents are where those two halves meet the bill. A PDF page is converted to an image *and* has its text extracted, and you pay for both — 1,500 to 3,000 text tokens per page plus the visual cost. A twenty-page A4 document at 150 dpi is 96,700 input tokens and 48p on Opus 5 where the same words as plain text are 20p, a factor of 2.42. Rendering at 100 dpi saves a third; rendering at 300 dpi saves nothing and costs nothing extra, because 200 and 300 dpi are downscaled to the identical page. And the same file is 2,835 visual tokens per page on Opus 5 and 1,551 on Haiku 4.5 — on a vision workload, choosing a model changes the token count of your input, not merely its price.

Finally, the two levers that are constantly confused. A `file_id` from the Files API cuts a ten-turn conversation's uploads from 32.0 MB to 3.2 MB and changes the token bill by exactly nothing. Prompt caching changes the token bill by 77.5 per cent at ten turns and 83.2 per cent at twenty — while making turn one 24.6 per cent dearer, which is why a single-request benchmark concludes that caching is a tax. Bandwidth and tokens are different problems. Reach for the right lever, and reach for both when a document will be asked more than one question.

KEY TAKEAWAYS

1. **Streaming is a delivery mode, not a different API.** Same endpoint, same body, same seven stop reasons; the response arrives as edits instead of a document.
2. **The server sends deltas; the client owns the message.** Reassembly is your job, and `strict` should be the default.
3. `index` **is the identity of a content block.** Every buffer is keyed by it, or two blocks silently become one.
4. `stop_reason` **arrives after the content it governs** — in `message_delta`, second from last, and unknown for 87 per cent of a stream's events.
5. **Render freely; act only after** `message_stop`**.** The model proposes, your system disposes, and a partial stream has not finished proposing.
6. **A 200 is not a promise.** `error` events arrive inside a successful response, and recovery differs on 4.6 and later because prefilling is gone.
7. **Fine-grained tool streaming trades validation for latency.** Guard the parse and return `INVALID_JSON` in an `is_error` tool result.
8. **Images cost patches, not pixels**: `ceil(w/28) × ceil(h/28)`, capped by a per-model tier, billed as ordinary input.
9. **A PDF page is text *and* an image.** 2.42× the price of the same words, and the pictures are what let Claude read the chart.
10. `file_id` **fixes bytes;** `cache_control` **fixes tokens.** They are different levers for different problems, and only one of them appears on your invoice.

17.44 Glossary

Term	Definition
Server-sent events (SSE)	A one-directional HTTP streaming format of `field: value` lines in blank-line-separated frames.
`message_start`	First event; a `Message` with empty `content` and `stop_reason: null`.
`content_block_start`	Opens a block at a given `index`; a `tool_use` block's `input` is `{}` here.
`content_block_delta`	Carries a `delta` that updates the block at `index`.
`content_block_stop`	Closes a block; the point at which accumulated tool JSON is parsed.
`message_delta`	Carries top-level changes — `stop_reason`, `stop_details` — and cumulative `usage`.
`message_stop`	Final event. The only proof the message finished.
`ping`	Keep-alive event carrying nothing; may appear anywhere.
`error` event	An error delivered inside a 200 response, e.g. `overloaded_error`.
`index`	A content block's position in the final `content` array; the key for every delta buffer.

Term	Definition
`text_delta`	Appends to a text block's `text`.
`thinking_delta`	Appends to a thinking block's `thinking`; absent under `display: "omitted"`.
`signature_delta`	The thinking block's integrity signature; arrives just before its `content_block_stop`.
`input_json_delta`	A `partial_json` **string** fragment of a tool block's `input`.
Fine-grained tool streaming	`eager_input_streaming: true` per tool; unbuffered, unvalidated tool input.
`fallback` content block	A start/stop pair with no deltas, emitted at a model boundary during server-side fallback.
Visual token	One 28×28-pixel patch of an image; the unit of image cost.
Resolution tier	A model's (long-edge, visual-token) limit pair: 2576/4784 or 1568/1568.
`image` block	Content block carrying a `base64`, `url` or `file` image source.
`document` block	Content block for a PDF or `text/plain` file; accepts `title`, `context`, `citations`.
`container_upload` block	Content block sending a file to the code execution tool.
Files API	Beta upload/list/retrieve/download/delete API returning a `file_id`. Free; 500 MB per file, 500 GB per organisation.
`file_id`	A reference to an uploaded file. Reduces payload bytes, not token cost.
Many-image request	More than 20 image or document blocks; a stricter per-image dimension limit applies.

17.45 Further Reading

Primary sources, all at `platform.claude.com/docs/en/`, verified 30 July 2026:

- `build-with-claude/streaming` — event types, delta types, full HTTP stream examples, error recovery by model generation.
- `agents-and-tools/tool-use/fine-grained-tool-streaming` — `eager_input_streaming`, the accumulation contract, the `INVALID_JSON` wrapper.
- `build-with-claude/vision` — source types, limits, the patch formula, the resolution tiers, the limitations list.
- `build-with-claude/vision-coordinates` — the exact resize-and-pad rule and a reference implementation; required reading before any bounding-box work.
- `build-with-claude/files` — endpoints, block types, storage limits, lifecycle, billing.
- `build-with-claude/pdf-support` — how a PDF is processed, page and size limits, the Bedrock Converse citations coupling.
- `build-with-claude/token-counting` — the free endpoint that removes the guesswork from every estimate in this chapter.

Where to go next in this book. Chapter 18 takes thinking seriously as an API surface — budgets, effort, signatures, interleaving — the topic this chapter touched only where streaming forced it. **Chapter 19** puts rate limits and quotas around everything here. **Chapter 23** returns to structured output, whose relationship with streaming is the `input_json_delta` accumulation you have just written. **Chapter 29** builds retry, timeout and backoff on top of section 17.7's `error` events, and **Chapter 30** develops prompt caching properly. **Chapter 34** revisits the `is_error` tool-result contract as a general pattern. **Chapter 52** treats the context budget as a design constraint, where a 96,700-token PDF stops being a cost and becomes a capacity question.